

Clase 155 — Malware en Android

Parte: **6 — Análisis de malware** · Fuente: *Learning Malware Analysis* y OWASP Mobile Security 🕒
Duración estimada: **120 min** · Nivel: **Avanzado**

Objetivo

Analizar aplicaciones Android maliciosas: entender el formato APK, descompilar el bytecode Dalvik a algo legible, revisar el manifiesto y los permisos, y reconstruir el comportamiento (robo de SMS/credenciales, overlays bancarios, spyware). El alumno aprenderá el flujo estático y dinámico con un emulador aislado.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Descomponer** un APK: manifiesto, permisos, DEX, recursos y librerías nativas.
2. **Descompilar** el bytecode a Smali/Java para leer la lógica.
3. **Identificar** permisos peligrosos y componentes exportados abusables.
4. **Reconocer** técnicas: overlays, accesibilidad, interceptación de SMS, C2.
5. **Analizar** dinámicamente en un emulador aislado con captura de tráfico.

Temas

#	Tema	Por qué importa
1	Estructura APK y Android Manifest	Punto de partida del análisis
2	Permisos y componentes exportados	Superficie de abuso
3	Bytecode Dalvik y Smali	El código real de la app
4	Descompilación a Java	Lectura rápida de la lógica
5	Abuso de Accessibility y overlays	Técnicas de troyanos bancarios
6	C2 y exfiltración	Robo de datos y control
7	Análisis dinámico en emulador	Comportamiento real

Definiciones y características

- **APK**: paquete ZIP de una app Android. Característica clave: **manifiesto + DEX + recursos**.
- **AndroidManifest.xml**: declara permisos y componentes. Característica clave: **mapa de capacidades**.
- **DEX / Dalvik bytecode**: código ejecutable de la app. Característica clave: **descompilable a Smali/Java**.
- **Accessibility abuse**: uso del servicio de accesibilidad para leer/controlar la UI. Característica clave: **control total** de la pantalla.
- **Overlay**: ventana falsa sobre una app legítima. Característica clave: **robo de credenciales** bancarias.

Herramientas y preparación

- **apktool** (desempaqueta y decodifica el manifiesto/Smali).
- **jadx** / **jadx-gui** (descompila DEX a Java).
- **MobSF** (análisis estático y dinámico automatizado).
- **Android Emulator/Genymotion** aislado, **Frida** (instrumentación), **mitmproxy** (tráfico TLS).

⚠ **Nota ética y de seguridad:** instala y ejecuta APKs maliciosos solo en un **emulador aislado** (sin cuentas reales, sin SIM, sin red hacia Internet salvo el proxy del lab). Nunca en tu teléfono personal. Trabaja únicamente con muestras autorizadas.

Laboratorio guiado

Estático:

1. Desempaqueta: `apktool d muestra.apk`. Revisa `AndroidManifest.xml`: permisos (`READ_SMS`, `SYSTEM_ALERT_WINDOW`, `BIND_ACCESSIBILITY_SERVICE`) y componentes `exported`.
2. Descompila a Java con `jadx-gui muestra.apk`. Localiza el `Application` /servicios y sigue el flujo desde `onCreate` /receivers de SMS.
3. Busca strings de C2 (URLs, endpoints), claves de cifrado y llamadas a `SmsManager`, `AccessibilityService`, `WebView` (overlays).
4. Corre **MobSF** para un informe automatizado y contrástalo con tu lectura manual.

Dinámico:

5. Instala el APK en el emulador aislado (`adb install muestra.apk`) con snapshot previo.
6. Configura **mitmproxy** como proxy y captura el tráfico C2 (instala el CA del proxy en el emulador). Observa registros y comandos.
7. Con **Frida**, engancha funciones de cifrado/red para revelar datos antes de cifrar y confirmar el C2.
8. Extrae IOCs (dominios, endpoints, permisos abusados) y mapea a ATT&CK Mobile. Restaura el snapshot.

Ejercicios

1. Enumera 6 permisos peligrosos y el abuso asociado a cada uno.
2. Explica cómo un troyano bancario usa overlays + accesibilidad.
3. Descompila una app y localiza el receptor de SMS.
4. Interpreta el informe de MobSF y prioriza hallazgos.
5. Captura tráfico C2 con mitmproxy y documenta un comando.
6. Usa Frida para revelar una cadena cifrada en runtime.

Reto verificable

Analiza un APK malicioso y entrega un **informe** con: permisos peligrosos, componentes exportados, técnica principal (overlay/SMS/spyware), C2 e IOCs, respaldado por descompilado y captura de tráfico. **Criterio de aceptación:** identificas la capacidad principal con la evidencia del código descompilado y capturas al menos un endpoint C2 en mitmproxy, sin haber usado un dispositivo real.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Instalar en el móvil personal	Riesgo real; usa solo emulador aislado
jadx falla al descompilar	Ofuscación/anti-descompilación; usa Smali de apktool
Sin tráfico TLS legible	Falta el CA del proxy o hay pinning; instala CA y bypass con Frida
Permisos "normales" pero malicioso	Revisa accesibilidad y APIs, no solo el manifiesto
Multi-DEX incompleto	La app tiene varios DEX; asegúrate de analizarlos todos

Preguntas frecuentes

 **¿Basta el análisis estático?** Da mucho (permisos, C2, lógica), pero la ofuscación y la carga dinámica exigen análisis dinámico con emulador y Frida.

 **¿Qué es el certificate pinning y cómo lo supero?** La app valida el certificado del servidor; para inspeccionar TLS en el lab se hace bypass con Frida en el emulador.

 **¿Por qué la accesibilidad es tan peligrosa?** Permite leer y controlar toda la interfaz: capturar credenciales, autorizar acciones y ocultar la actividad al usuario.

Referencias

- OWASP Mobile Security Testing Guide. <https://owasp.org/www-project-mobile-app-security/>
- jadx. <https://github.com/skylot/jadx>
- apktool. <https://apktool.org>
- MobSF. <https://github.com/MobSF/Mobile-Security-Framework-MobSF>
- MITRE ATT&CK Mobile. <https://attack.mitre.org/matrices/mobile/>

Siguiente clase

Clase 156 - Reglas YARA para detección